

Entregable 1: Diagrama de clases UML con evidencia de comparación entre diseño propio y diseño sugerido por IA

Sistema de Gestión de Fincas e Inmuebles

Integrantes (Grupo 1)

- Fabio Perez Ruiz
- Maria Alejandra Ramos Naizir
- Karla Tom Cabarcas

<i>Elemento analizado</i>	<i>Diseño inicial del grupo</i>	<i>Sugerencia de la IA</i>	<i>Decisión final del grupo</i>	<i>Justificación</i>
Clase Inmueble	Clase base para representar propiedades	Convertirla en clase abstracta	Se convirtió a abstracta	Se identificó que Piso, Local y Edificio comparten atributos como dirección, descripción, código postal, precio de alquiler y disponibilidad. Por esta razón se decidió crear una clase abstracta Inmueble para evitar repetir información en varias clases y organizar mejor el modelo. Además, un inmueble genérico no tendría sentido como objeto independiente dentro del sistema.
Clase Alquiler	No existía inicialmente	Crear una clase independiente	Se agregó la clase alquiler	Inicialmente se pensó manejar el alquiler directamente desde Inmueble, pero esto hacía que la clase tuviera demasiadas responsabilidades. Se creó la clase Alquiler para representar la relación entre un Inquilino y un Inmueble, además de almacenar información propia del alquiler como fechas y estado.
Relación Edificio – Piso/Local	No estaba representada	Agregar relación entre clases	Se agregó la relación	El enunciado establece que un edificio puede contener pisos y locales. Por esta razón se agregó una relación entre Edificio y las clases Piso y Local, permitiendo representar la estructura real de los inmuebles administrados por la inmobiliaria.
Clase Factura	Se manejaban atributos separados (agua, gas, electricidad, etc.)	Representar cada factura como un objeto independiente	Se simplificaron los atributos para representar Factura como un objeto independiente	Se observó que una factura puede representar distintos conceptos y no únicamente servicios específicos. Por esta razón se decidió usar atributos generales como concepto, proveedor y costo, permitiendo registrar

				diferentes tipos de gastos sin necesidad de crear muchos atributos fijos.
Atributos particulares en Piso y Local	Las clases incluían atributos específicos como numeroPiso y tipoEspacio	Se sugirió evaluar si el atributo numeroPiso debía eliminarse de Local, y tipoEspacio de Piso para representar mejor las características específicas de cada tipo de inmueble	Se mantendrán ambos atributos en sus respectivas clases	Se decidió mantener los atributos numeroPiso y tipoEspacio como características particulares de las clases Piso y Local porque el enunciado establece que estos tipos de inmuebles pueden registrar información específica adicional a la heredada desde Inmueble. El atributo nPiso permite identificar la ubicación de un piso dentro de un edificio, mientras que tipoEspacio permite diferenciar la función de un local, como oficina, local comercial o bodega. Con esta decisión se buscó representar de manera más precisa las características propias de cada tipo de inmueble sin sobrecargar la clase general Inmueble.
Clase MovimientoBancario	Se manejaban categorías como atributos independientes (ingresosAlquiler, reformas, reparaciones, etc.)	Reemplazar categorías fijas por atributos generales como tipoMovimiento	Se modificó la estructura de MovimientoBancario	Se identificó que cada movimiento bancario debe representar una operación financiera individual dentro del sistema. Por esta razón se decidió eliminar atributos específicos para cada tipo de movimiento y reemplazarlos por atributos generales que permitan registrar distintos conceptos, como ingresos por alquiler, pagos de servicios, reformas o reparaciones, manteniendo un diseño más flexible y organizado.
Relación entre InmobiliariaAPP e Inquilino	La relación no estaba claramente definida	Mantener una colección de inquilinos administrada por la aplicación	Se agregó relación mediante agregación	El sistema necesita registrar, almacenar y consultar múltiples inquilinos. Por esta razón se decidió que la clase principal InmobiliariaAPP mantenga una colección de inquilinos para centralizar la administración de la información del sistema.
Relación entre Alquiler, Inquilino e Inmueble	La relación entre las clases estaba definida por agregación	Utilizar asociaciones en lugar de agregación	Se cambiaron las relaciones por asociaciones simples	Se decidió utilizar asociaciones entre Alquiler y las clases Inquilino e Inmueble debido a que el alquiler únicamente representa la relación entre

				ambas entidades dentro de un periodo determinado. Ni el inmueble ni el inquilino dependen de la existencia del alquiler para existir dentro del sistema.
Clase GestorArchivos	No existía una clase para almacenar los datos y no perderlos	Crear una clase separada para manejar archivos	Se agregó la clase	El sistema requiere guardar y recuperar información de inmuebles, inquilinos y alquileres. Se decidió crear una clase independiente para organizar el manejo de archivos y evitar mezclar esta responsabilidad con la lógica principal de la aplicación.